Capstone 04 – Team Guide From Zero

Nov 2, 2026 · @bhavesh

This guide takes a team of five from an empty laptop to a finished Databricks project, one day at a time. It assumes you know nothing about Azure or Databricks. Every step says where to click, what to type and what you should see when it works.

1. Start here

The project in plain words

A retail company records every sale as a row in a file. The files are messy: some rows are repeated, some values are missing, some amounts are absurdly large, and a sale's status (completed, reversed and so on) changes later in a separate file. The company cannot trust its reports.

Your team builds a clean, trusted data system in Databricks and proves it works. The system answers four questions:

- **Net sales:** how much money came from sales that are completed?
- **Gross margin:** how much of that is profit?
- **Return rate:** what share of sales were reversed?
- **Available stock:** how many units of each product are left?

You are marked on proof as much as on code, so you will collect screenshots and numbers all week. A checklist at the end says exactly which.

The data you get

File or table	What it holds	Size
sale_batch_01 to 12	One row per sale event	11,540 rows
status_updates	Later status changes for sales	520 rows
secondary_events	Alerts and reviews linked to sales	310 rows
store_id, product_id	Lists of stores and products	110 and 150 rows

The sales files hold 40 exact duplicates, 12 missing sales_channel values, 35 deliberately huge amounts, and a new column in batches 10 to 12. These are planted on purpose. Your job is to find them, handle them and explain what you did.

Words you will hear

Word	What it means here
Azure	Microsoft's cloud. You use it to store the files.
Storage account, container	A cloud hard drive, and a folder-like box inside it.
Databricks	The tool where you write and run the data code.
Workspace	Your Databricks website, with your notebooks and tables.
Notebook	A page of code split into cells. You press Run to execute it.
Serverless	Computing power Databricks starts for you, so you do not set up a machine.
Unity Catalog	Databricks' filing system. It has catalogs, which hold schemas, which hold tables and volumes.
Volume	A folder in Unity Catalog that points at your files in Azure.
Delta table	A table that keeps its history, so you can look at earlier versions.
Bronze, Silver, Gold	Raw copy, cleaned copy, final business tables.
Auto Loader	A Databricks feature that reads new files once and remembers which it has read.
CDC (change data capture)	The list of status changes that must be applied to sales in order.
Workflow (job)	A saved chain of notebooks that runs by itself.
KPI	A key business number, such as net sales.
MLflow	A tool that records a machine-learning experiment.
Genie	Databricks' feature for asking questions in plain English.

Who does what

Every member must be able to explain the whole pipeline, because the client may ask anyone. The roles decide who leads each part.

Member	Role	Leads
1	Technical lead	Design documents, Unity Catalog, final sign-off
2	Ingestion engineer	Azure storage, Auto Loader, Bronze
3	Quality engineer	Silver, quality rules, status changes
4	Analytics engineer	Gold tables, dashboard, MLflow, Genie
5	DataOps engineer	Workflow, tests, Git, performance, runbook

2. Before you start

What you need

- [ ] A laptop with Chrome or Edge, and a stable internet connection
- [ ] A Microsoft account that can sign in to portal.azure.com, with permission to create resources and assign roles
- [ ] Access to a Databricks workspace on Azure with Unity Catalog, a serverless SQL warehouse and serverless notebook compute. Ask your instructor for the link.
- [ ] A GitHub account (free) for Day 4
- [ ] Python 3 installed, to run the local tests on Day 4. Check by typing python --version in a terminal.
- [ ] Git installed. Check by typing git --version.
- [ ] The code package **retail_capstone_team_pack.zip** from your team lead, and the original project zip with the data files. Unzip both on every laptop.

Decide these together on day one

The team shares **one** Azure setup and **one** Databricks catalog. Building five copies wastes the week. Agree on these names and write them down in a shared note:

Item	Name to use	Why
Resource group	rg-retail-capstone	The folder that holds everything in Azure; deleting it removes everything
Storage account	Lowercase letters and numbers only, globally unique, for example retailcap plus your team name	Azure rejects names that someone else has taken

Item	Name to use	Why
Container	retail-landing	The folder inside the storage account
Catalog	retail_capstone_team	The code package uses this name; use another only if you must, and then edit 00_config
Schema	lakehouse	Fixed in the code
Job	retail_capstone_pipeline	The Day 4 Workflow

If a name is taken in a shared workspace, add a short suffix to the catalog name, for example retail_capstone_team5, and change it in notebook 00_config and in every SQL file.

Working rules

1. Do not edit the source CSV files. Bronze must hold exactly what the source sent.
2. Do not delete anything in the shared Databricks workspace that you did not create.
3. Never put passwords or keys in notebooks. If you use a secret, store it in a Databricks secret scope.
4. Take a screenshot after every result marked **Evidence**. Save them in a shared folder named by day, for example Day3.
5. Keep a decision log: one line each time the team chose something (what, why, who). It becomes part of your report.
6. Run notebooks in order, and run each cell group only once unless a step says to rerun.
7. If something fails, copy the full error text before you change anything. The Common errors section near the end lists the ones we have already seen.

3. Day 1, part A: create the Azure storage

Goal: a cloud folder that holds the source files, and a permission that lets Databricks read it. **Who:** the Ingestion engineer does it, the Technical lead watches. Everyone else follows on screen and then repeats steps 1 and 2 to see the result. **Time:** about 45 minutes.

Step A1. Sign in to Azure

1. Open **portal.azure.com** in Chrome or Edge.

2. Sign in with the Microsoft account your instructor gave you. If you must make your own, go to azure.microsoft.com/free, choose **Start free**, and follow the sign-up. It needs a phone number and a card for identity checks. Some free accounts cannot run Databricks until upgraded to pay-as-you-go; only upgrade if a later step fails and your instructor agrees.
3. You should see the Azure home page with a search bar at the top.

Step A2. Create a resource group

A resource group is a folder for everything in this project. Deleting it at the end removes all of it.

1. In the search bar type **Resource groups**, open it and click **Create**.
2. Choose your **Subscription**. Name it **rg-retail-capstone**. Pick a **Region** near you, for example Central India, and use the same region for everything below.
3. Click **Review + create**, then **Create**.

Step A3. Create the storage account

1. Search **Storage accounts** and click **Create**.
2. On the **Basics** tab choose your resource group. Type a storage account name: lowercase letters and numbers only, 3 to 24 characters, unique worldwide. Pick the same region.
3. Set **Performance** to Standard and **Redundancy** to Locally-redundant storage (LRS). This is the cheapest.
4. Open the **Advanced** tab and tick **Enable hierarchical namespace**. This turns an ordinary storage account into ADLS Gen2, which Databricks needs. If you forget it, delete the account and start again.
5. Click **Review + create**, then **Create**, then **Go to resource** when it finishes.

Step A4. Create the container and folders

1. In the left menu of the storage account click **Data storage**, then **Containers**.
2. Click **+ Container**, name it **retail-landing**, and click **Create**.
3. Open **retail-landing**. Click **Add Directory**, type **landing** and save.
4. Open **landing** and add four more directories, one at a time: **primary_events**, **status_updates**, **secondary_events**, **reference**.

Step A5. Upload the data files

The data files are in the project zip under datasets.

Open this Azure folder	Upload these files from your laptop
primary_events	All 12 files, sale_batch_01.csv to sale_batch_12.csv (datasets/adls/primary_events)
status_updates	status_updates.csv (datasets/adls/status_updates)
secondary_events	secondary_events.csv (datasets/adls/secondary_events)
reference	store_id.csv and product_id.csv (datasets/azure_sql_extracts)

For each folder: click the folder name, click **Upload**, click **Browse for files**, select the files, and click **Upload**. Click **landing** in the path at the top to go back.

Careful: the code package also has a folder called reference. It holds Python code and must **not** be uploaded here. Only the two CSV files go into the Azure folder named reference.

You should see: primary_events holds 12 files; status_updates, secondary_events hold 1 each; reference holds exactly 2 CSV files.

Step A6. Create the Access Connector

This is the identity Databricks uses to read your storage.

1. Search **Access Connector for Azure Databricks** and click **Create**.
2. Choose the same resource group and region. Name it **ac-retail-capstone**.
3. Click **Review + create**, then **Create**.

Step A7. Give the connector permission on the storage

1. Open your **storage account**, then **Access Control (IAM)** in the left menu.
2. Click **Add**, then **Add role assignment**.
3. On the **Role** tab search **Storage Blob Data Contributor**, select it and click **Next**.
4. On the **Members** tab choose **Managed identity**, then click **+ Select members**.
5. In the panel set **Managed identity** to **Access Connector for Azure Databricks**, click **ac-retail-capstone** so it appears under Selected members, and click **Select**.
6. Click **Next** twice, then **Review + assign**, and confirm.
7. Check **Role assignments** shows the connector with Storage Blob Data Contributor. It can take a minute to appear.

If you get a permission error here, your account is not allowed to assign roles. Ask your instructor or subscription owner to do this one step.

Step A8. Copy the connector's Resource ID

1. Open **ac-retail-capstone** and look at the **Overview** page.
2. Find **Resource ID** and click the copy icon next to it. If you only see JSON, use **JSON View** and copy the value after "id".
3. Paste it into your team note. It starts with /subscriptions/ and ends with /accessConnectors/ac-retail-capstone.

Evidence for Day 1: screenshot the container with its four folders, each folder's file list, and the role assignment.

4. Day 1, part B: connect Databricks and load Bronze

Goal: Databricks can see your files, and the raw data sits in Bronze tables. **Who:** the Ingestion engineer runs it; the Technical lead creates the catalog. **Time:** about 60 minutes.

Step B1. Open Databricks

Open the workspace link from your instructor and sign in. In the left menu you should see Workspace, Catalog, Jobs & Pipelines, Compute, SQL Editor, Dashboards and Genie.

Step B2. Create the storage credential

This tells Databricks to use the connector from Step A6.

1. Click **Catalog** in the left menu.
2. Click the **gear icon** next to the word Catalog at the top of the left panel. A menu opens.
3. Under **Connect** click **Credentials**, then **Create credential**.
4. Choose **Azure Managed Identity**. Name it **cred_retail**. Paste the Resource ID you copied in Step A8 into **Access connector ID**. Click **Create**.

If the Create button is greyed out, you are not a metastore admin. Ask your instructor to create the credential.

Step B3. Create the external location

1. Open the same gear menu and click **External locations**, then **Create external location**.
2. Name it **loc_retail**.

3. For **URL** type: abfss://retail-landing@YOURSTORAGEACCOUNT.dfs.core.windows.net/ and replace YOURSTORAGEACCOUNT with your storage account name. It must start with abfss:// (not https://), keep the @ sign, and end with a slash.
4. Choose **cred_retail** as the credential, then click **Create** or **Next**.
5. Click **Test connection**.

You should see: Success for Read, List, Write, Delete, Path Exists and Hierarchical Namespace Enabled. Two **File Events** checks may fail with a long message about missing roles. This is harmless: file events only make Auto Loader a little faster. Click **Force create** if it offers it.

Step B4. Create the catalog, schema and volume

You do this once, in the SQL Editor, not in a notebook.

1. Click **SQL Editor** in the left menu. At the top choose a **Serverless** SQL warehouse.
2. Paste the four statements below, replacing YOURSTORAGEACCOUNT. Run them **one at a time**: select a statement and click **Run selected**.

```
CREATE CATALOG IF NOT EXISTS retail_capstone_team
  MANAGED LOCATION 'abfss://retail-
landing@YOURSTORAGEACCOUNT.dfs.core.windows.net/catalog_root';

CREATE SCHEMA IF NOT EXISTS retail_capstone_team.lakehouse;

CREATE EXTERNAL VOLUME IF NOT EXISTS retail_capstone_team.lakehouse.landing
  LOCATION 'abfss://retail-
landing@YOURSTORAGEACCOUNT.dfs.core.windows.net/landing';

LIST '/Volumes/retail_capstone_team/lakehouse/landing/primary_events';
```

You should see: the LIST statement returns 12 rows, one per batch file. If it returns nothing, the files are not in the right folder (see Step A5). If CREATE CATALOG says it cannot create a catalog, you may lack permission: ask your instructor.

These statements are also saved in sql/setup_catalog.sql in the code package.

Step B5. Give your teammates access

The person who created the catalog owns it. Each teammate needs permission to use it. Run once per teammate, with their email as it appears in Databricks:

```
GRANT ALL PRIVILEGES ON CATALOG retail_capstone_team TO `teammate@email.com`;
GRANT READ VOLUME, WRITE VOLUME ON VOLUME
retail_capstone_team.lakehouse.landing TO `teammate@email.com`;
```

If your instructor says not to grant access, work from one shared login and take turns.

Step B6. Import the notebooks

1. Click **Workspace** in the left menu, then your home folder.
2. Click **Create**, then **Folder**, name it **retail_capstone**, and open it.
3. Click the three dots or **Create**, then **Import**. Drag in the six files from the notebooks folder of the code package: 00_config.py, 01_bronze_ingest.py, 02_silver.py, 03_gold.py, 04_validation.py, 05_mlflow_forecast.py.
4. Keep all six in the **same folder**. Notebook 00_config is loaded by the others, and they will not find it if it is somewhere else.
5. Open **00_config**. It should say CATALOG = "retail_capstone_team". Change it only if your team chose a different catalog name, and then change the path in LANDING too. You do **not** need to run 00_config yourself; the other notebooks load it.

Step B7. Run notebook 01: load Bronze

1. Open **01_bronze_ingest**.
2. At the top right click the compute selector and choose **Serverless**.
3. Click **Run all**. It takes one to three minutes. Each cell shows a green tick when it finishes.

What it does: Auto Loader reads the three ADLS folders into **bronze_sale_events**, **bronze_status_cdc** and **bronze_secondary_events**, adding the source file name and ingest time to each row. Then it copies the two reference CSVs into **bronze_store_id** and **bronze_product_id**, labelled CSV_EXTRACT_SIMULATION.

You should see at the bottom: rows = 11540 and keys = 11500. That is 11,540 rows, of which 40 are duplicates.

Step B8. Look at what you made

In **Catalog**, open retail_capstone_team, then lakehouse, then Tables. You should see five Bronze tables. Click one and open **Sample Data** to see rows.

Evidence for Day 1: screenshot the external location test, the LIST result, the Bronze count (11540 and 11500), and the table list. Do not claim a live Azure SQL connection: the reference tables came from CSV files, and the report must say so.

5. Day 1, part C: the design documents and Git

Goal: the written design the client signs off before more is built. **Who:** the Technical lead owns it; everyone reviews. **Time:** about 2 hours. These documents are part of your marks, and the code package does not contain them, so your team writes them.

What each document is

Document	One-line meaning	How long
BRS (business requirements)	What the client wants and how you will know it is done	1 page
HLD (high-level design)	A picture of the whole system and how data flows	1 page and a diagram
LLD (low-level design)	Table by table: what each notebook reads and writes	1 to 2 pages
ERD	A diagram of the Gold tables and how they join	1 diagram
Source-to-target mapping	For each source column, where it ends up	1 table
Data dictionary	What every column in the final tables means	1 table

Write them in a shared document. Plain sentences and tables are fine.

BRS: what to put in it

- **Objective:** reduce stockouts and returns, and support stock availability and revenue quality.
- **Stakeholders:** operations manager, analytics lead, data governance owner, platform operations, client product owner.
- **Functional needs:** ingest sale events, apply status changes, publish Gold tables with net sales, gross margin, return rate and available stock.
- **Non-functional needs:** restart-safe pipelines, least-privilege access, traceable source metadata, freshness reporting, failure alerts.
- **Acceptance:** expected-versus-actual reconciliation; no duplicate keys in current-state tables; documented exceptions; a repeatable end-to-end run; a client walkthrough.
- **Out of scope:** live transaction decisions, production model serving, enterprise CI/CD, real customer personal data.

HLD: the system in one line

Draw this as boxes and arrows (PowerPoint, draw.io or the whiteboard in your document):

ADLS files and Azure SQL tables, then Unity Catalog volume, then Bronze, then Silver, then Gold, then SQL warehouse, dashboard and Genie. A Databricks Workflow runs the notebooks in order. Git holds the code.

Under the picture write two sentences about each layer: what goes in, what comes out.

LLD: one row per notebook

Notebook	Reads	Writes	Main logic
01_bronze_ingest	CSV files in the volume	bronze_sale_events, bronze_status_cdc, bronze_secondary_events, bronze_store_id, bronze_product_id	Auto Loader with checkpoints; adds source file and ingest time
02_silver	Bronze tables	silver_sale_events, silver_rejected_records, silver_status_history, silver_sale_current	Typing, quality rules, deduplication, status changes in order
03_gold	Silver tables	fact_inventory_ledger, fact_sale_activity, fact_status_transitions, mart_sale_daily, mart_available_stock, dim_store, dim_product	Business rules and KPIs
04_validation	Bronze to Gold tables	a result table	Compares actual numbers with expected ones and fails if they differ
05_mlflow_forecast	mart_sale_daily	an MLflow run	A small demand forecast baseline

ERD: how the Gold tables connect

Draw one box per table and a line for each join:

- fact_sale_activity joins dim_store on **store_id** and dim_product on **product_id**.

- fact_inventory_ledger joins fact_sale_activity on **sale_id**, and dim_product on **product_id**.
- fact_status_transitions joins fact_sale_activity on **sale_id**.
- mart_sale_daily is built from fact_sale_activity by **sale_date**.
- mart_available_stock is built from fact_inventory_ledger by **product_id**.

Source-to-target mapping (main columns)

Source column	Silver column	Rule applied	Gold use
sale_id	sale_id	Must exist; one row per sale_id	Key of fact_sale_activity
event_timestamp	event_ts	Converted to a timestamp	sale_date, trend
ingest_timestamp	ingest_ts	Converted to a timestamp	Tie-breaker for duplicates
store_id	store_id	Must exist in store list	Joins dim_store
product_id	product_id	Must exist in product list	Joins dim_product
sales_channel	sales_channel	Missing becomes UNKNOWN	Channel breakdown
order_status	order_status, then current_status	Replaced by the latest status change	Decides net sales and returns
sales_amount	sales_amount	Number above 0; above 1000 flagged	net_sales, gross_margin
quantity	quantity	Number above 0	Ledger movements, stock
change_timestamp, new_status	history rows	Applied in order	fact_status_transitions

Data dictionary (main Gold columns)

Column	Table	Meaning
net_sales	fact_sale_activity, mart_sale_daily	Sales amount of sales whose current status is COMPLETED
returned_amount	same	Sales amount of sales whose current status is REVERSED
gross_margin	same	Net sales times (1 minus 0.70)
return_rate	mart_sale_daily	REVERSED sales divided by COMPLETED plus REVERSED sales
is_outlier	silver_sale_current	True if sales_amount is above 1,000
signed_qty	fact_inventory_ledger	Minus quantity for a SALE, plus quantity for a RETURN
available_stock	mart_available_stock	Opening stock (1,000) plus the sum of signed_qty

Git: start the repository

Do this on one laptop, then the others clone it.

1. Create a free account at github.com and click **New repository**. Name it retail-capstone and keep it empty.
2. Open a terminal in the unzipped code package folder and run, one line at a time, replacing the address with yours:

```
git init
git add .
git commit -m "Initial capstone pipeline"
git branch -M main
git remote add origin https://github.com/YOURNAME/retail-capstone.git
git push -u origin main
```

3. Add each teammate under the repository's **Settings, Collaborators**.
4. Rule for the week: nobody changes main directly. Make a branch (git checkout -b feature/short-name), push it, and open a pull request that another member reviews. A pull request checklist is: runs without errors, numbers unchanged, no passwords in the code.

Evidence for Day 1: the six documents, the architecture picture, the first Git commit, and the Bronze screenshots from part B.

6. Day 2: clean the data into Silver

Goal: one clean row per sale, bad rows set aside with a reason, and every status change applied in the right order. **Who:** the Quality engineer leads. **Time:** about 3 hours. **What the client asks:** typed and deduplicated data; a rule catalogue with IDs; a quarantine table that explains each rejected row; status changes that never overwrite newer ones; unit tests.

Step 1. Understand the rules before you run anything

Silver applies a numbered list of checks. Read it so you can explain it to the client.

Rule	What it checks	What happens	Rows in this data
DQ-01	sale_id is present	Row goes to the rejected table	0
DQ-02	Timestamps can be read	Rejected	0
DQ-03	sales_amount is a number above 0	Rejected	0
DQ-04	quantity is a number above 0	Rejected	0
DQ-05	order_status is a known value	Rejected	0
DQ-06	store_id exists in the store list	Rejected	0
DQ-07	product_id exists in the product list	Rejected	0
DQ-08 / 09	The sale_id appears more than once	Keep the latest copy, record the others as rejected	40
DQ-10	sales_channel is empty	Keep the row, fill UNKNOWN, flag it	12
DQ-11	sales_amount is above 1,000	Keep the row, flag as an outlier	34

A tie-breaker decides which duplicate stays: the latest ingest time, then the file name, then the row number. The same input always gives the same answer.

Step 2. Run notebook 02_silver

1. Open **02_silver** in your retail_capstone folder, make sure **Serverless** is selected, and click **Run all**.
2. Wait about a minute. Every cell should get a green tick.

What it builds:

- **silver_sale_events:** the clean sales, one row per sale_id.
- **silver_rejected_records:** every row set aside, with rule_id, reason, source file and row.
- **silver_status_history:** every valid status change (a Delta MERGE, so rerunning does not add rows twice).
- **silver_sale_current:** each sale with its latest status in a column called current_status.

Step 3. Check the results

Open the **SQL Editor** and run these one at a time:

```
USE CATALOG retail_capstone_team;
USE SCHEMA lakehouse;

SELECT count(*) AS silver_rows FROM silver_sale_events;
SELECT rule_id, count(*) AS n FROM silver_rejected_records GROUP BY rule_id;
SELECT count(*) AS history_rows FROM silver_status_history;
SELECT current_status, count(*) AS n FROM silver_sale_current GROUP BY
current_status;
```

Query	You should see
silver_rows	11500
rejected by rule	DQ-08/09 with 40
history_rows	520
statuses	COMPLETED 2938, PENDING 2714, FAILED 2749, OPEN 2748, CANCELLED 188, REVERSED 163

If a number is different, do not move on. Copy the numbers and the first error into the team chat.

Step 4. Follow one sale through the status changes

```
SELECT sale_id, order_status, current_status, status_changed_ts
FROM silver_sale_current WHERE sale_id = 'SALE-0000001';
```

The sale arrived as **FAILED** and ended as **COMPLETED**, because the status file said so. Use this sale in your client presentation.

Step 5. The inventory ledger rule

The client wants sales, returns and stock movements combined into one signed ledger. The files have no returns or stock columns, so the team derives movements from the status. This rule is applied in the Gold notebook on Day 3; agree on it now and write it in the decision log.

Current status	Ledger rows	Effect on stock
COMPLETED	one SALE row	minus the quantity
REVERSED	one SALE row and one RETURN row	minus, then plus the quantity (net zero)
OPEN, PENDING, FAILED, CANCELLED	none	no change

Say in your report that this is a derived rule, not source data.

Step 6. Run the unit tests on your laptop

These tests check the same rules with small made-up examples (valid, invalid, duplicate, late and out-of-order rows). They need no Databricks.

1. Open a terminal (Command Prompt or PowerShell on Windows) in the unzipped code package folder.
2. Run: pip install pandas pytest
3. Run: pytest tests -v

You should see: 10 passed. If pip is not found, try python -m pip install pandas pytest.

Evidence for Day 2: the four query results, the rejected table, the SALE-0000001 result, the written rule table, and the 10 passed screenshot. Record before and after counts: 11,540 in, 40 rejected, 11,500 out.

7. Day 3: build Gold and prove the numbers

Goal: the four KPIs in tables the business can use, plus proof that they match the clean data. **Who:** the Analytics engineer leads; the Quality engineer reviews the reconciliation. **Time:** about 3 hours. **What the client asks:** facts and dimensions at a documented grain; KPIs for net sales, gross margin, return rate and available stock; Gold checked against Silver; reruns that do not double count; Delta history explained.

Step 1. Run notebook 03_gold

1. Open **03_gold**, choose **Serverless**, and click **Run all**.
2. It prints nothing when it works. Look for a green tick on every cell.

What it builds. "Grain" means what one row stands for:

Table	One row per	Used for
fact_sale_activity	sale_id	Net sales, returns, margin per sale
fact_status_transitions	sale and status change	Status history
fact_inventory_ledger	sale and movement type	Signed stock movements
mart_sale_daily	day (32 days)	Trend, KPI cards
mart_available_stock	product (150)	Stock
dim_store, dim_product	store, product	Lookup of names and regions

The KPI definitions used:

- **Net sales** = sum of sales_amount where current_status is COMPLETED.
- **Gross margin** = net sales times 0.30. The 70% cost is an assumption, because the files have no cost column.
- **Return rate** = REVERSED sales divided by (COMPLETED plus REVERSED sales).
- **Available stock** = 1,000 opening units (assumed) plus the signed ledger movements for each product.

Step 2. Run notebook 04_validation

1. Open **04_validation** and click **Run all**.
2. It compares eight numbers from your tables with the expected values.

You should see a table with every row true:

Metric	Value
bronze_rows	11540
silver_rows	11500
rejected_dups	40
net_sales_total	1517371.37
gross_margin_total	455211.43
ledger_rows	3264
stock_total	111530.46
outliers	34

The notebook also stops with an error if keys are duplicated or a store or product is missing. Screenshot the table.

Step 3. Run the SQL checks

Open the SQL Editor and paste the contents of **sql/day3_checks.sql** from the code package. It begins with these two lines, which tell the editor which catalog to use:

```
USE CATALOG retail_capstone_team;
USE SCHEMA lakehouse;
```

The file has three parts. Run each part on its own with **Run selected**.

Part 1, one table of 13 checks. Every row must say true: Silver, fact and mart all have 11,500 rows; no duplicate sale_id; 32 days; net sales 1,517,371.37 in both Silver and the mart; gross margin 455,211.43; 3,264 ledger rows; stock 111,530.46; zero missing stores and zero missing products.

Part 2, the incremental update. It is a MERGE that recalculates the last eight days of mart_sale_daily. Run it **twice**, then run Part 1 again. Everything must still be true. This proves a rerun does not double count. The result says 8 rows updated and 0 inserted.

Part 3, Delta history and the query plan.

- DESCRIBE HISTORY mart_sale_daily lists version 0 (the table was created) and the MERGE versions after it.
- SELECT … VERSION AS OF 0 returns 32 rows. This is time travel: the table as it was at the start.

- EXPLAIN FORMATTED shows how Databricks will run a query. Look for a scan of fact_sale_activity followed by a grouping step.
- The skew query returns 76, 76.7 and 77. Each product has almost the same number of rows, so no product overloads the engine.

Step 4. Take the lineage screenshot

1. Click **Catalog** and open retail_capstone_team, then lakehouse, then Tables, then **mart_sale_daily**.
2. Click the **Lineage** tab, then **See lineage graph**.
3. You should see fact_sale_activity feeding mart_sale_daily. Click the small **+** at the left of a box to follow the chain back to Silver. If nothing shows, wait ten minutes and refresh.

Step 5. Explain the July 1 spike

When you chart daily sales, 1 July is about five times higher than any other day. It is not an error. Seven of the flagged outlier sales fall on that day. In sql/day5_genie_checks.sql the last query shows 224,746.90 with them and 29,842.90 without. The outliers stay in the KPIs and appear in the exception table, and the client may choose to exclude them.

Evidence for Day 3: the validation table, the 13-check table before and after the double MERGE, DESCRIBE HISTORY, the version 0 count, the query plan, the skew result, the lineage graph, and a short grain table. State clearly that gross margin and available stock rest on two assumptions.

8. Day 4: automate, break it on purpose, recover

Goal: the whole pipeline runs by itself in the right order, survives a failure, and is safe, measured and documented. **Who:** the DataOps engineer leads. **Time:** about 4 hours. **What the client asks:** a Workflow with retries and alerts; proof that a rerun and a failure leave the data correct; restricted access; a measured performance test; monitoring; Git history; a runbook.

Step 1. Create the Workflow (job)

A Workflow runs your notebooks one after another so nobody has to click Run all four times.

1. Click **Jobs & Pipelines** in the left menu, then **Create**, then **Job**.
2. Click the job's name at the top (it says New Job and a date) and type **retail_capstone_pipeline**. Press Enter.

3. A form for the first task opens. Fill it in:

Field	Task 1
Task name	ingest_bronze
Type	Notebook
Source	Workspace
Path	retail_capstone / 01_bronze_ingest
Compute	Serverless
Retries	2 (click the pencil icon; the default says at most 3 times)

4. Click **Create task**.
5. Click **Add task**, choose **Notebook**, and add the other three the same way. Check **Depends on** is set to the task above it.

Task name	Notebook	Depends on
silver	02_silver	ingest_bronze
gold	03_gold	silver
validate	04_validation	gold

The graph should show four boxes in a chain: ingest_bronze, silver, gold, validate.

6. **Failure email:** in the right-hand panel scroll down past Job health configuration to **Notifications**. Click **Add notification**, enter your email, tick **Failure** and save. If you cannot find it, use the Notifications row inside each task's form instead.

Step 2. Run it

1. Click **Run now** at the top right.
2. Open the **Runs** tab and click the run that appears.
3. Wait five to eight minutes. Boxes go from grey to blue (running) to green (succeeded).

You should see: four green boxes. Because validate checks the numbers and stops with an error if any differ, a green run means the pipeline and its checks passed. Screenshot the graph, then click **validate** and screenshot its output table. Call this **Run 1**.

Step 3. Replay test: run the same input again

This proves that a rerun does not create duplicates or inflate the numbers.

1. Go back to the job page (click retail_capstone_pipeline at the top) and click **Run now** again. Add no new files.
2. When all four boxes are green, open validate and screenshot its table. Call this **Run 2**.
3. Compare: every number must be identical to Run 1.
4. Extra proof in the SQL Editor: select count(*) as bronze_rows, count(distinct sale_id) as keys from bronze_sale_events must still give 11540 and 11500. Auto Loader remembers which files it has read.

Step 4. Failure test: break it and recover

1. Open the notebook **02_silver**. Add a new cell at the very top with one line: raise Exception("DEMO FAILURE")
2. Run the job. ingest_bronze goes green, silver fails (it retries twice first, so wait a few minutes), and gold and validate are skipped. Screenshot the red graph.
3. Go back to 02_silver and **delete** the cell you added. Do not forget this, or every later run fails.
4. Open the failed run and click **Repair run**, then confirm with **Repair run**. Only silver, gold and validate run again.
5. When all are green, open validate and screenshot the table. The numbers must match Run 1.

This is your recovery proof: after a failure and a repair, nothing was lost or duplicated.

Step 5. Security check

Run in the SQL Editor and screenshot:

```
SHOW GRANTS ON SCHEMA retail_capstone_team.lakehouse;
SHOW GRANTS ON TABLE retail_capstone_team.lakehouse.mart_sale_daily;
```

This lists who can use the data. In a shared workspace do not grant access to everyone. If you want to show a grant, give read-only access to one named teammate:

```
GRANT USE CATALOG ON CATALOG retail_capstone_team TO `teammate@email.com`;
GRANT USE SCHEMA, SELECT ON SCHEMA retail_capstone_team.lakehouse TO
`teammate@email.com`;
```

Also screenshot the job's Run as setting (it shows your user). Note in the report that a service principal was not available in the shared workspace, if that is true for you.

Step 6. Performance test (OPTIMIZE)

Paste **sql/day4_checks.sql** into the SQL Editor. It does the following in order:

1. Turns off the result cache, so every run does real work.
2. Runs one query three times (a sum of net_sales by product). After each run open **Query History** in the left menu and write down its duration.
3. Runs OPTIMIZE on fact_sale_activity, which tidies the table's files.
4. Runs the same query three more times and writes down the durations.

Put the six numbers in a table: before and after. **Report them as measured.** The table has only about 11,500 rows, so the difference will probably be small or none. Never claim an improvement you did not measure.

Step 7. Monitoring

The same file runs two monitoring queries. Screenshot them:

- **Freshness:** the latest ingest time in bronze_sale_events against the current time.
- **Rejected counts:** rows set aside by rule in silver_rejected_records.

Also screenshot the job's **Runs** tab, which keeps the history of every run and how long each took.

Step 8. Git: branch, pull request, merge

1. On a laptop with the cloned repository run git checkout -b feature/day4-hardening
2. Open docs/RUNBOOK.md and add your six timing numbers at the bottom.
3. Run git add . then git commit -m "Add Day 4 benchmark results" then git push -u origin feature/day4-hardening
4. On GitHub click **Compare and pull request**, then **Create pull request**. A teammate reviews it and clicks **Merge pull request**.
5. Screenshot the merged pull request and the commit list.

Step 9. The runbook

The file docs/RUNBOOK.md lists what to do when something breaks: ingest fails, validation fails, duplicate keys appear, a KPI looks inflated, data is stale. Read it together, add what you learned in Step 4, and keep it in Git.

Evidence for Day 4: Run 1 and Run 2 graphs and validation tables, the failed run, the repaired run and its table, SHOW GRANTS, the six timings and the OPTIMIZE result, the monitoring results, the pytest output, and the merged pull request.

9. Day 5: dashboard, MLflow, Genie and acceptance

Goal: show the client the results, check them against the SQL, and finish with a clear sign-off request. **Who:** the Analytics engineer leads the dashboard, MLflow and Genie; the Technical lead leads UAT and the presentation. **Time:** about 5 hours.

Step 1. MLflow: a small forecast

The client asked for a simple forecast of daily sales volume, recorded in MLflow so anyone can see exactly how it was trained. It is a baseline, not a production model.

1. Open **05_mlflow_forecast**, choose **Serverless** and click **Run all**.
2. The notebook takes the daily table, trains on the earlier 80% of days and tests on the later 20%. Training only on the past means the model never sees the future, which is called avoiding leakage.
3. Click **Experiments** in the left menu, open the newest run called daily_demand_baseline, and screenshot the parameters and the two metrics: mae_model and mae_naive_lag7. A lower mae is better. Compare your model with the simple baseline that repeats last week's value, and say honestly which is better.

There are only 32 days of data, so the test period is short. Say so in your notes. If logging the model fails with an artifact or registry message, copy the error to the team and move on after one attempt; this part carries few marks.

Step 2. The dashboard

A dashboard shows the KPIs on one page. You first create datasets (queries), then add charts that use them.

1. Click **Dashboards** in the left menu, then **Create dashboard**. Name it **Retail KPIs**.
2. Open the **Data** tab and click **Add SQL dataset**. Add the seven datasets below one at a time. Paste each query, click **Run**, and rename the dataset to the name shown. All seven queries are in **sql/day5_dashboard_datasets.sql**.

Dataset name	What it returns
kpis	net_sales, gross_margin, return_rate
stock	total_stock, lowest_product_stock
trend	sale_date, net_sales, returned_amount
by_channel	sales_channel, net_sales
low_stock	the ten products with least stock

Dataset name	What it returns
rejects	rejected rows by rule_id
outliers	the ten largest outlier sales

3. Open the canvas tab (it says Untitled page). Add these visualizations with **Add a visualization**, or type a request in the assistant box such as "Build counters for net sales, gross margin, return rate and total stock from the datasets kpis and stock, a line chart of net sales by date from trend, bar charts from by_channel and low_stock, and tables from rejects and outliers":

- four **Counter** cards: net sales, gross margin, return rate (as a percentage), total available stock
- a **Line** chart: sale_date on the x axis, net_sales on the y axis
- a **Bar** chart of net sales by channel
- a **Bar** chart of the ten lowest-stock products
- two **Tables**: rejected records by rule, and the top ten outlier sales

4. Click **Publish** at the top right and screenshot the dashboard.

Check every card against the SQL. These are the right values:

Card	Value
Net sales	1,517,371.37
Gross margin	455,211.43
Return rate	5.26%
Total available stock	111,530.46

Two things to expect and explain: the line chart starts with a spike on 1 July (the outliers), and the channel chart has a tiny UNKNOWN bar (the 12 missing channel values).

Step 3. Genie: ask questions in plain English

1. Click **Genie Agents**, then **New**.
2. Add only these tables: mart_sale_daily, fact_sale_activity and mart_available_stock. Click **Create**.
3. Ask the three questions below, one at a time. Screenshot each answer, and open its source or code view to see the SQL Genie wrote.

4. Run the matching SQL from **sql/day5_genie_checks.sql** and compare. Write down whether they agree. If one differs, report it honestly. That is what a grounded check means.

Question to type	Check
Which five days had the highest net sales?	Top five by net_sales. First is 2026-07-01 with 224,746.90, then 07-10, 07-28, 07-11, 07-09
What is net sales by sales channel?	Sum by channel; B2B, ONLINE and STORE are each roughly 470,000 to 550,000, plus a tiny UNKNOWN
Which 10 products have the lowest available stock?	Ten lowest available_stock values, all roughly 590 to 600

If your workspace has no Genie, write that down and use the SQL results as your validation. The brief allows this.

Step 4. Acceptance tests (UAT)

Copy this table into your report and add one screenshot or result per row. Mark Pass or Fail honestly.

Test	What to show	Expected
TC01	Bronze count and duplicates	11,540 rows, 40 duplicates, 11,500 unique
TC02	Missing and extreme values	12 handled, 34 flagged
TC03	Replay Run 2 against Run 1	Same numbers
TC04	SALE-0000001	FAILED changes to COMPLETED, history kept
TC05	A bad record	Goes to rejected table with a reason (covered by the unit tests)
TC06	Failed run and repair	Same numbers after repair
TC07	Store and product joins	0 unmatched keys
TC08	Gold KPIs against independent query	Equal
TC09	Dashboard and Genie against SQL	Equal
TC10	Before and after OPTIMIZE	Six honest timings

Also build a **traceability matrix**: one row per client requirement, with the notebook or job that delivers it, the test that proves it and the evidence file. Use the table in section 11 as a start.

Step 5. The client presentation (20 minutes)

Minutes	Slide or demo	Who
2	The business problem and why stock and returns matter	Technical lead
3	Architecture: Bronze, Silver, Gold, in one picture	Ingestion engineer
3	Follow SALE-0000001 from the source file to Gold	Quality engineer
2	Data quality: duplicates, nulls, outliers and the rejected table	Quality engineer
2	KPI numbers and how they were reconciled	Analytics engineer
3	The dashboard, then MLflow and Genie	Analytics engineer
5	Live failure and recovery demo, then access and Git history	DataOps engineer
5	Risks, assumptions, limitations and the sign-off request	Technical lead

Practise once with the clock. End by asking the client to accept the release.

Evidence for Day 5: the MLflow run, the published dashboard with the checked values, three Genie answers beside their SQL, the UAT table, the traceability matrix, and the slides.

10. Common errors and how to fix them

These are real problems met while building this project. Always copy the full error text before changing anything.

What you see	Why it happens	What to do
Metastore storage root URL does not exist when creating a catalog	The workspace has no default storage, so a catalog needs a location	Create the catalog in the SQL Editor with MANAGED LOCATION (Step B4). The notebooks do not create catalogs

What you see	Why it happens	What to do
Expression monotonically_increasing_id is not supported with streaming	An older copy of notebook 01 adds a row number to a stream	Use the latest code package. Otherwise delete that line from 01 and add it in 02 where Bronze is read as a table
PATH_NOT_FOUND for reference/store_id.csv	The two CSV files are not in the reference folder, or wrong files (Python code) were uploaded there	In Azure, delete the wrong files and upload store_id.csv and product_id.csv into landing/reference
LIST returns no rows	Files are in a different folder, or the volume path is wrong	Check the folder names exactly: primary_events, status_updates, secondary_events, reference
Test connection shows File Events failed	Optional roles for faster file detection are missing	Ignore it. Read, List, Write, Delete and Path Exists are what matter. Click Force create
Test connection says the request is not authorized	The role assignment has not taken effect yet, or was not made	Wait three minutes and test again. Check the connector has Storage Blob Data Contributor on the storage account
The connector is missing from the list when assigning the role	It was not created, or is in another subscription	Create ac-retail-capstone first, in the same resource group, then try again
You cannot assign roles	Your account lacks permission	Ask your instructor or subscription owner to do that one step
Create credential or Create catalog is greyed out	You are not an admin in this workspace	Ask your instructor to create it, or to give you the permission
Storage account name is already taken	Names are shared worldwide	Add digits or your team name
Overwriting a reference table fails after switching the source	The column types changed from text to date	Use the latest notebook 01, which sets overwriteSchema

What you see	Why it happens	What to do
A SQL file shows many results after Run all	Each statement gives its own result	Use the left and right arrows beside Results to move through them
MERGE says 8 rows updated, 0 inserted	This is correct: it recalculates the last eight days	Nothing to fix. Totals must stay the same
Every later run fails after the failure test	The line that raises the error is still in 02_silver	Delete the test cell and use Repair run
The Notifications setting is not visible	The right panel is long	Scroll it down, zoom the browser out, or use the Notifications row inside each task
Lineage tab is empty	It lags behind new tables	Wait ten minutes, refresh, and try fact_sale_activity
Genie's answer differs from the SQL	Genie chose another table or wording	Report the difference honestly and say which is right
The 1 July spike on the chart	Seven flagged outlier sales	Not an error. Explain it with the query in sql/day5_genie_checks.sql
You cannot find Serverless compute	The workspace has a different compute setup	Ask your instructor which cluster or warehouse to attach

11. Final checklist, traceability and cleanup

What to submit

- [ ] BRS, HLD, LLD, ERD, source-to-target mapping and data dictionary
- [ ] GitHub repository link, with a merged pull request and commit history
- [ ] Bronze, Silver and Gold tables in Databricks, with the table list screenshot
- [ ] Quality report: the rule table, the rejected records and before and after counts
- [ ] CDC and reconciliation results: validation table, 13-check table, double MERGE result
- [ ] Automated test log: the "10 passed" output
- [ ] Workflow evidence: Run 1, Run 2 (replay), the failed run and the repaired run

- [ ] Performance comparison: six honest timings and the OPTIMIZE output
- [ ] Security evidence: SHOW GRANTS and the Run as setting
- [ ] Monitoring: freshness, rejected counts and job run history
- [ ] Dashboard (published) with each card checked against SQL
- [ ] Genie validation: three answers next to their SQL
- [ ] MLflow run: parameters, metrics and the run screenshot
- [ ] Runbook, release notes and the known-limitations list
- [ ] Client presentation

Traceability matrix (start from this)

Client requirement	Delivered by	Proved by
Ingest the files and keep raw history	Notebook 01, Bronze tables	TC01, TC06
Restart-safe pipeline	Auto Loader checkpoints, the Workflow with retries	TC03, TC06
No duplicate keys in the current-state tables	Notebook 02 deduplication and MERGE	TC01, TC03
Documented exceptions	silver_rejected_records, rule table	TC02, TC05
Latest status plus history	silver_sale_current and silver_status_history	TC04
Valid store and product keys	Rules DQ-06, DQ-07 and the dimension tables	TC07
KPIs that reconcile	Notebook 03 and notebook 04	TC08
Dashboard and Genie agree with SQL	Dashboard and Genie space	TC09
Measured performance	OPTIMIZE benchmark	TC10
Least-privilege access and traceable lineage	Grants, Unity Catalog lineage	Security evidence

Known limitations to state

- The files hold no returns, stock, cost or opening-stock data. The ledger is derived from status, opening stock is assumed to be 1,000 units per product, and the cost ratio is assumed to be 70%. Ask your instructor to confirm these.

- The reference tables came from CSV files. Say so, unless you completed a live Azure SQL load and can show the proof.
- Only 34 of the 35 injected extreme values can be caught by the amount rule.
- The workspace was shared, so access testing was limited.
- With 11,500 rows, performance results are small and the OPTIMIZE gain may be negligible.

Cleanup after you are marked

1. Do not delete anything in the shared Databricks workspace that belongs to someone else.
2. In the Azure portal open **Resource groups**, open **rg-retail-capstone**, and click **Delete resource group**. This removes the storage account, the connector and any SQL database, and stops all charges. Do it only after every screenshot is saved and the project is submitted.
3. Remove your own catalog and notebooks in Databricks only if your instructor asks.